CSIT 112 Lab 5: Parallelizing Image Processing (Box Blur Filter)

Introduction: How a Blur Filter Works

A blur filter is one of the simplest and most common image-processing techniques. Its goal is to smooth details and reduce noise by averaging the color values of neighboring pixels. The effect makes edges softer and the image less sharp.

Blurring works by sliding a small matrix (called a kernel or filter) across the image. For each output pixel, the algorithm examines its surrounding neighborhood (for example, a 3×3 window), averages the color values, and stores the average in the corresponding output pixel.

3×3 box blur kernel: 1/9 × [[1 1 1], [1 1 1], [1 1 1]]

Because each output pixel can be computed independently from the original input image, different rows of the output image can be processed in parallel. This makes the box blur a good example for exploring data parallelism and multithreading.

Learning Goals

- Understand data-parallel workloads in image processing.
- Practice Java parallel programming using ExecutorService.
- Understand the distinction between tasks and worker threads in a fixed-size thread pool.
- Compare sequential and parallel execution performance using execution time, speedup, and efficiency.

Prerequisites

- JDK 8 or later installed.
- Folders: images/input/ and images/output/.
- Example input image: aroma.jpg.
- Reference programs: EdgeDetectionSeq and EdgeDetectionParallel.

What You'll Build

Implement a 3×3 box blur filter in two ways:

1. Sequential version (provided).

2. Parallel version using ExecutorService.

Starter Code

- BoxBlurSeq.java – complete sequential version.
- BoxBlurParallel.java – skeleton with TODOs.

Lab Steps

Part A — Sequential Baseline

1. Compile and run BoxBlurSeq.java.

2. Record the execution time and examine the output image.

Question: Why does the program skip the 1-pixel border? *(Hint: Consider what happens when the 3×3 kernel reaches the edge of the image.)*

Part B — Parallel Execution

Modify BoxBlurParallel.java to:

- Use ExecutorService with a fixed-size thread pool.
- Create one task for each image row and submit the tasks to the thread pool.
- Use the thread-pool size to control the number of worker threads. The number of tasks is not the same as the number of threads.
- Ensure that all submitted tasks have completed before recording the final execution time and saving the output image.

Experiments

1. Run the parallel program using 1, 2, 4, 8, 16, and 32 worker threads. Record the execution time for each configuration.

2. Compare the output images from the sequential and parallel versions. Verify that the parallel version produces the expected result.

3. Calculate speedup and parallel efficiency for each thread count.

4. Create a graph showing execution time versus number of threads and discuss the performance trend.

Deliverables

- Completed BoxBlurParallel.java source code.
- Two output images: image_blur_seq.jpg and image_blur_parallel.jpg.
- A 1–2 page report containing performance measurements, a graph, calculations, and discussion. Use the report template below.

Grading Rubric (50 pts)

Component	Points
Correct box blur implementation	15
Correct parallel implementation using ExecutorService	20
Performance experiments and discussion	5
Clear report and analysis	10

Lab 5 Report Template

Lab Title: Parallelizing Image Processing — Box Blur Filter

Student Name: _____

Student ID: _____

Date: _____

Course / Section: _____

1. Introduction

Briefly explain the purpose of this lab and what a blur filter does. Address the following:

- What is the goal of blurring?
- How does the 3×3 box blur kernel work?
- Why is this image-processing operation a good example for parallelization?

(Write your paragraph here.)

2. Implementation Overview

Describe what you implemented. Discuss both program versions:

- Sequential version (BoxBlurSeq.java).
- Parallel version (BoxBlurParallel.java).
- How the work was divided into tasks (one task per image row).
- How the fixed-size thread pool executes many row tasks using a limited number of worker threads.
- Any implementation challenges you encountered.

(Write your paragraph here.)

3. Experimental Setup

Describe your test environment and configuration:

- CPU model and number of available processor cores.
- Java version.
- Image dimensions.
- Thread counts tested: 1, 2, 4, 8, 16, and 32.
- How execution time was measured.

(Write your paragraph here.)

4. Results and Performance Analysis

4.1 Execution Time

Version	1 Thread	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
Sequential	_____ ms	—	—	—	—	—
Parallel	_____ ms	_____ ms	_____ ms	_____ ms	_____ ms	_____ ms

Fill in your measured execution times in milliseconds.

4.2 Performance Graph

Create a graph showing parallel execution time versus number of worker threads (1, 2, 4, 8, 16, and 32). Include clear axis labels and units.

(Insert your graph here.)

4.3 Output Comparison

Compare the output images from the sequential and parallel versions. Were there any visual differences? If so, explain what you observed.

(Write your paragraph here.)

4.4 Speedup and Efficiency

Speedup = T*sequential* / T*parallel*

Efficiency = Speedup / Number of Threads

Threads	Parallel Time (ms)	Speedup	Efficiency
1	_____	_____	_____
2	_____	_____	_____
4	_____	_____	_____
8	_____	_____	_____
16	_____	_____	_____
32	_____	_____	_____

Discuss the performance trend. Did execution time continue to improve as more threads were added? How did efficiency change? Explain possible reasons, considering the number of processor cores and the overhead of managing parallel tasks.

(Write your analysis here.)

5. Conclusion

Summarize the key findings of the lab. Address the following:

- What did you learn about data parallelism and parallel programming?
- How effective was parallelization for this image-processing task?
- What did you observe when the number of worker threads increased?
- What improvements or alternative task-decomposition strategies might you consider?

(Write your paragraph here.)